Taurus Database: How to be Fast, Available, and Frugal in the Cloud

Alex Depoutovitch, Chong Chen, Jin Chen, Paul Larson, Shu Lin, Jack Ng, Wenlin Cui, Qiang Liu, Wei Huang, Yong Xiao, Yongjun He
Huawei Research Canada

arXiv:2412.02792v1 [cs.DB] 3 Dec 2024

ABSTRACT

Using cloud Database as a Service (DBaaS) offerings instead of on-premise deployments is increasingly common. Key advantages include improved availability and scalability at a lower cost than on-premise alternatives. In this paper, we describe the design of Taurus, a new multi-tenant cloud database system. Taurus separates the compute and storage layers in a similar manner to Amazon Aurora and Microsoft Socrates and provides similar benefits, such as read replica support, low network utilization, hardware sharing and scalability. However, the Taurus architecture has several unique advantages. Taurus offers novel replication and recovery algorithms providing better availability than existing approaches using the same or fewer replicas. Also, Taurus is highly optimized for performance, using no more than one network hop on critical paths and exclusively using append-only storage, delivering faster writes, reduced device wear, and constant-time snapshots. This paper describes Taurus and provides a detailed description and analysis of the storage node architecture, which has not been previously available from the published literature.

CCS CONCEPTS

• **Information systems** → **DBMS engine architectures**.

KEYWORDS

cloud databases, availability, reliability, architecture

ACM Reference Format:
Alex Depoutovitch, Chong Chen, Jin Chen, Paul Larson, Shu Lin, Jack Ng, Wenlin Cui, Qiang Liu, Wei Huang, Yong Xiao, Yongjun He. 2020. Taurus Database: How to be Fast, Available, and Frugal in the Cloud. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD'20), June 14–19, 2020, Portland, OR, USA.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3318464.3386129

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD'20, June 14–19, 2020, Portland, OR, USA
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-6735-6/20/06…$15.00
https://doi.org/10.1145/3318464.3386129

1 INTRODUCTION

As companies move their applications to the cloud, demand for cloud-based relational database service (DBaaS) is growing rapidly. Amazon, Microsoft, Alibaba, and other cloud providers all offer such services. Most DBaaS offerings were initially based on traditional monolithic database software, essentially just running databases on (virtual) machines in the cloud, using either local storage or cloud storage. Although simple to implement, this approach cannot provide what customers want from a cloud database service [12]. From a customer's point of view, an ideal database service should be highly available, require no maintenance, and scale up and down automatically with database size and workload. It should also deliver high performance, be low cost, and users should pay only for resources actually used (pay-as you-go). These goals can't be achieved by running the same old database software in the cloud - the system must be redesigned from the ground up.

A good DBaaS architecture must simultaneously provide durability, scalability, performance, availability, and cost-effectiveness. Having three persistent copies of data distributed across different hosts is usually considered sufficient to ensure durability [5]. Scalability can be provided by distributing compute and storage resources across hosts. While distributing a database across multiple hosts improves durability and scalability, it can adversely affect availability. Using many hosts increases the probability of some hosts becoming unavailable, potentially resulting in the whole database becoming unavailable. Methods, such as quorum-based replication and eventual consistency, can increase availability, but both have drawbacks. Quorum-based replication may need a higher replication factor than what is required for pure data durability, increasing the cost. Eventual consistency is unacceptable for many existing applications.

In this paper, we present Taurus, a relational database designed specifically for cloud environments. Taurus builds on ideas from previous work and improves on them in several respects. Taurus separates compute and storage layers in a

fashion similar to Aurora [24] and, like Socrates, separates the concepts of availability and durability [2]. Taurus offers similar benefits, such as read replica support, fast fail-over and recovery, hardware sharing, and scalability to 128TB.

The Taurus compute layer consists of a single master (primary) and multiple read-only replicas (secondaries). Data is divided into pages that are partitioned across multiple storage nodes. All update transactions are handled by the master. The master ships log records to the storage layer over the network. The storage layer writes them to reliable storage, stores database pages, applies received log records to bring pages up to date, and responds to page read requests.

Taurus introduces several innovations, helping it achieve high availability and high performance at a low cost. Our first contribution is a replication and recovery algorithm that achieves high availability with a replication factor no higher than what is required for durability and without sacrificing performance or strong consistency guarantees. This algorithm enables Taurus to achieve nearly 100% availability for writes when considering uncorrelated storage failures. With only 3-way data duplication required for durability, Taurus achieves availability comparable to the 6-way quorum replication used by Aurora and better than the 3-way quorum replication used by POLARDB [4].

A key observation is that data access patterns for the database log and database pages are significantly different. Database logs are used solely for data durability and are written and read sequentially. Log writes are frequent and performance is critical, but log reads are required only during generally infrequent failures, thus read performance is of lesser concern. As logs are necessary for durability, they need strong consistency guarantees.

Unlike database logs, database pages are accessed randomly. Page read performance is critical, but write latency is less important because when a modified page needs to be written, the changes are already durable in the log. Database pages are versioned, and versioning can be used to provide strong consistency. Because of such differences, it is inefficient to use the same storage for logs and pages. Taurus separates log storage (Log Stores) from page storage (Page Stores). It uses different distribution and replication algorithms for log and page data. Each algorithm is optimized for its particular requirements.

Log reads are infrequent, and most log records are discarded without ever being read. A log record does not depend on other log records. Log records don't need to be written to specific Log Store servers, but rather can be written to any available Log Stores in a large pool, provided that the number of available Log Stores is enough for durability. In practice, a DBaaS deployment contains hundreds to thousands of nodes. A database can acknowledge writes as long as there are three available storage nodes, which for practical purposes should always be the case considering only uncorrelated failures.

In contrast, database pages are updated by applying log records and the previous version of a page is required to produce the next one. This requires a page to be assigned to specific Pages Store servers. Thus, the availability of a page depends on the availability of the specific Page Stores responsible for the page. However, in contrast to the traditional quorum-based replication algorithms, Taurus does not need a majority or even a plurality of Page Stores to be involved in each read or write operation. Strongly consistent log servers and page versioning can be used when deciding which Page Store replica is the most current one. If the page version is known, Page Stores can be eventually consistent, which is known to improve service availability [10]. The lack of a quorum requirement allows Taurus to reduce the number of replicas to that required for durability.

Another key factor for a successful DBaaS is high performance. In distributed systems, performance significantly depends on the number of operations on critical paths that cross network boundaries. Our second contribution is a set of novel architectural choices that enable Taurus to achieve higher performance. Our tests show that Taurus can achieve up to 200% higher throughput than MySQL 8.0 running with local storage. Taurus supports multiple read replicas and keeps the replica lag under 20ms even under high load. Most performance-critical operations in Taurus, such as writing logs and reading pages, require only one interaction between network-connected hosts.

To avoid having the storage layer become a bottleneck, Taurus improves storage layer performance in two ways. First, the separation of Log Stores from Page Stores reduces the load on Page Stores. Second, the organization of Page Stores is optimized around a "the log is the database" model, never modifying data in place, and performing append-only writes. This approach has multiple advantages, improving write performance by 2-5 times, while simultaneously reducing device wear and consequently, the cost of the service. Also, append-only writes simplify consistency algorithms and snapshot generation. Finally, the separation of the storage layer into Log and Page Stores, allows Taurus read replicas to receive updates directly from Log Stores, bypassing the master and avoiding the master becoming a bottleneck. Taurus distributes resource consumption necessary to support read replicas evenly across the scalable storage layer.

The third contribution of this paper is a detailed description of the inner workings, performance optimizations, and design compromises in the storage layer. Although the available literature [2, 24] describes many optimizations done in the compute layer, it lacks details about storage organization and the rationale for design decisions.